

In standard software engineering, you create these UML diagrams during the **System Design phase**—which sits exactly between finalizing your SRS and writing your first line of code. You use the SRS as your absolute source of truth to draw the blueprints. Once the blueprints (UML) are approved, you build the software.

Here is the logical sequence for creating these diagrams, grouped by their phase in the design process, and exactly where the **Class Diagram** fits into your workflow.

Phase 1: High-Level Analysis (Behavioral)

You start by modeling human interaction and business logic before worrying about technology.

1. **Use Case Diagram:** Create this first. It maps your user classes (Learner, Content Admin, Guest) directly to what they can do on the platform.
2. **Activity Diagram:** Create this second. It takes a single use case (e.g., "Complete a Lesson") and charts the step-by-step flow from start to finish, mapping out the logic gates for correct or incorrect answers.

Phase 2: High-Level Architecture (Structural)

Now you define the big, structural blocks that will execute those behaviors. 3. **Component Diagram:** Create this third. This maps out your React application, the CDN dependencies, your local storage strategy, and the external JSON file. It defines the physical boundaries of your system based on your constraint to avoid a build toolchain.

Phase 3: Detailed Design (Structural & Behavioral)

This is where you get granular, mapping exactly how the code will be written and how data will flow. 4. **Class Diagram (or Data/Component Model):** Create this fourth. It bridges the gap between the high-level components and the actual code. * *Note for React:* Since modern React relies heavily on functional components rather than traditional Object-Oriented Programming (OOP) classes, your "Class Diagram" will actually model two things: 1. Your **React Component Hierarchy** (e.g., Roadmap contains multiple StageCard components). 2. Your **Data Models** (the exact JSON schema for your curriculum and the structure of your local storage state). 5. **State Machine Diagram:** Create this fifth. As discussed, you map the strict lifecycles of entities, such as a stage moving from *locked* to *in progress* to *completed*. 6. **Sequence Diagram:** Create this last. It is the most detailed behavioral diagram, charting the exact time-ordered sequence of function calls and data fetches (e.g., User clicks "Submit" -> UI triggers validation -> Engine calculates XP -> Storage updates state).

Designing the Class / Data Diagram

Since you are storing your curriculum in a separate curriculum.json file to fulfill the requirement that content updates require no code redeployment, your Class/Data diagram must clearly define the structure of this JSON object.

Let's test your data modeling skills for this specific step.

Your SRS requires that the eight stages are presented sequentially, that stage titles and descriptions are bilingual (Sinhala and English), and that each stage contains a list of lessons.

If you are mapping out the "Stage" entity in your Class/Data diagram, what specific attributes (or JSON keys) must that object contain to satisfy all of those SRS requirements?